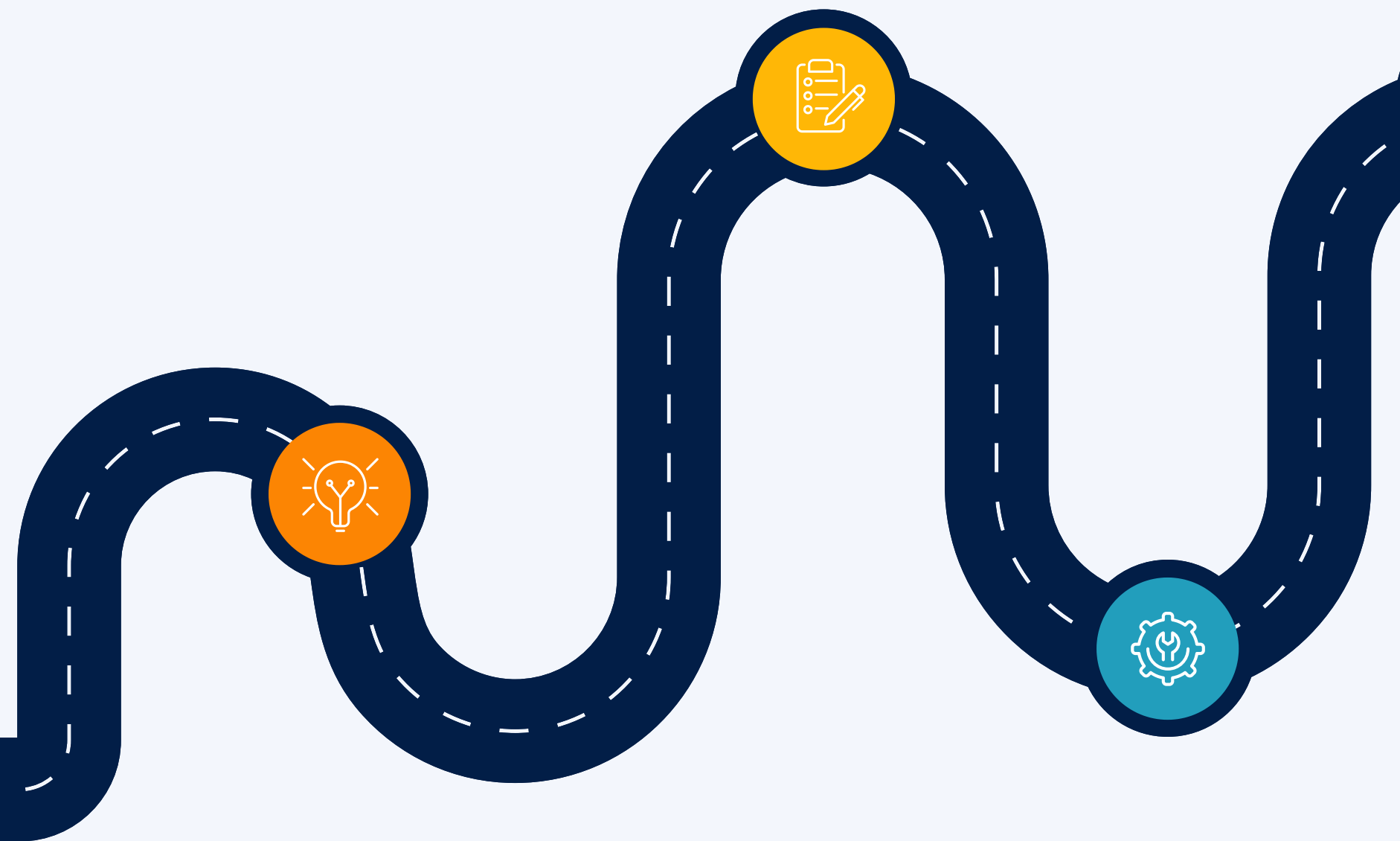


COMPETÊNCIA GERAL

# **CRIAR SERVIÇOS WEB COM REST**

Projeto WEB com REST





COMPETÊNCIA GERAL

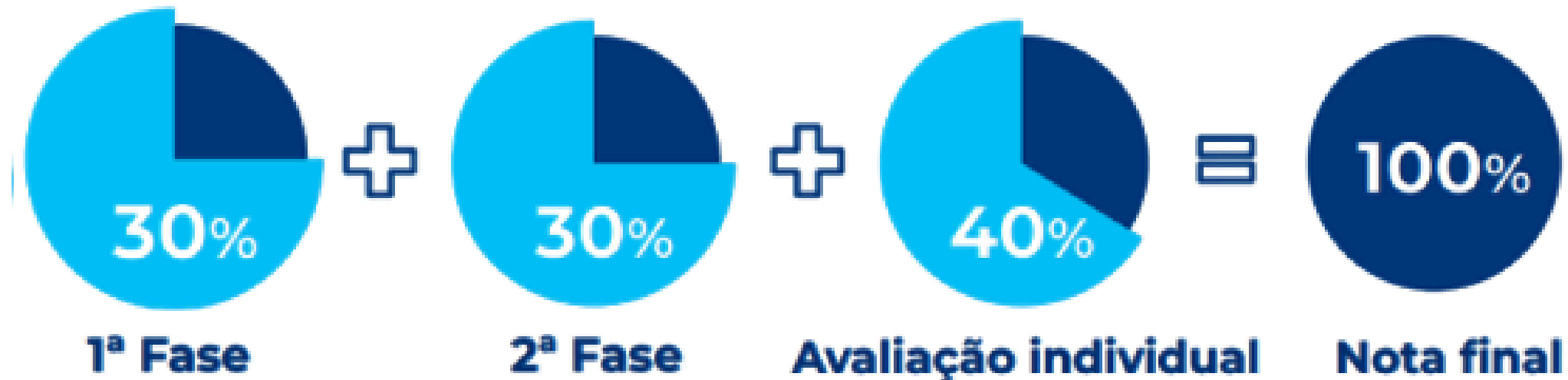
# **CRIAR SERVIÇOS WEB COM REST**

PRODUTO/ARTEFATO DO SABER

O aluno deverá entregar um sistema WEB com backend funcional feito em NodeJS e MongoDB e frontend feito em React



## Modelo Avaliativo



O estudante será **APROVADO** se:

Frequência  $\geq$  a 75% e Nota Final  $\geq$  a 7,0

O estudante **NÃO TEM DIREITO A FAZER A AVALIAÇÃO FINAL** se:

Nota do Projeto < 4,0

O estudante fará **AVALIAÇÃO FINAL** se:

Frequência  $\geq$  a 75%, Nota do Projeto  $\geq$  4,0 e "Nota do Projeto+Avaliação Individual" < 7

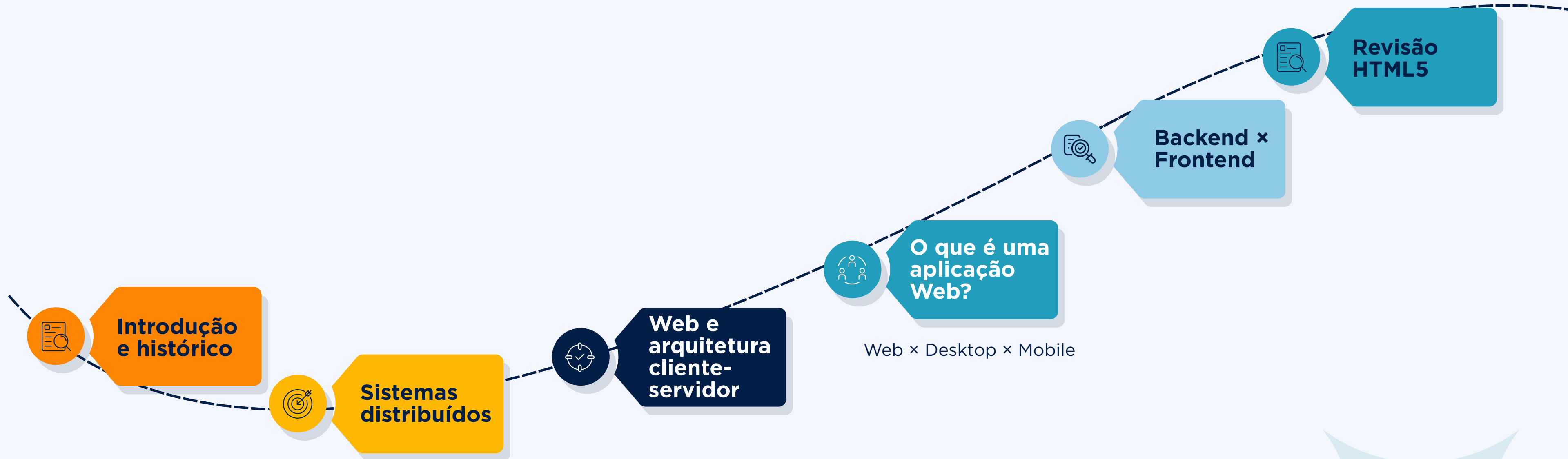
O estudante será **APROVADO** se:

A Avaliação Final + Nota do Projeto  $\geq$  6,0

**\*O processo avaliativo é detalhado no Regulamento Institucional de Avaliação para os cursos de graduação.**

# CONTEÚDOS

Encontro 01



# INTRODUÇÃO E HISTÓRICO





# 1989-1991

Web nasce (HTTP/HTML)



**Tim Berners-Lee  
propõe a Web  
no CERN**

A Web surge para compartilhar documentos científicos em um laboratório europeu de pesquisa em física



**HTTP: protocolo  
simples de  
requisições/  
respostas**

O HyperText Transfer Protocol define como cliente e servidor conversam.  
Ex.: o navegador pede “/index.html” e o servidor responde com o arquivo.



**HTML:  
linguagem de  
marcação para  
hipertexto**

O HyperText Markup Language define estrutura (títulos, parágrafos, links).  
(títulos <h1>,  
parágrafos <p>,  
links <a>)



**Primeiros  
servidores e  
navegadores  
(WorldWideWeb,  
Mosaic)**

Tudo muito  
texto-centrado,  
sem estilo  
complexo ou  
lógica no cliente.



**Ideia central:  
documentos  
ligados por  
links**

A inovação é o hiperlink, conectando páginas.  
Hipertexto → ligar documentos com links clicáveis para navegar.

# 1989-1991

Impacto inicial

**Modelo  
cliente-servidor  
popularizado**

**Navegação  
stateless (cada  
requisição é  
independente)**

A ausência de estado embutido simplificou a escala, mas exigiu estratégias posteriores para sessão/estado.

**Conteúdo  
predominantemente  
estático**

O foco era publicar e descobrir informação, não aplicações ricas.

**Padrões abertos  
→ adoção global**



# 1995

CSS e JavaScript surgem

**CSS: separa  
estilo do  
conteúdo  
(cores, layout)**

CSS permite  
consistência visual  
sem poluir o HTML

**JavaScript  
(Netscape):  
interatividade  
no navegador**

JS habilita validações,  
menus, efeitos.

**Começo do  
DOM  
manipulável**

**Passo da Web  
estática para  
dinâmica**

Juntos, abrem caminho  
para interfaces mais  
ricas sem recarregar a  
página inteira.



# 1995

Consequências técnicas

**DHTML:  
HTML + CSS + JS**

**Manipulação do  
DOM e eventos**

**Compatibilidade  
entre  
navegadores vira  
desafio**

**Padrões do W3C  
ganham  
importância**

Cada navegador implementava o DOM de forma diferente, gerando "cross-browser hell". Isso motiva bibliotecas futuras (ex.: jQuery) e o movimento de padronização.

# 2008-2014

HTML5/CSS3

**HTML5:** <video>, <audio>, <canvas>, semântica (<header>, <section>)

**APIs Web:** LocalStorage, Geolocation, History, WebWorkers

**CSS3:** transições, transformações, animações, media queries

**Base para** responsividade e apps ricas

Navegadores passam a suportar mídia nativa, gráficos e armazenamento local. CSS3 viabiliza layout fluido para múltiplas telas e animações sem plugins (adeus Flash).

# 2008-2014

Mudanças de arquitetura

**Mais lógica no cliente**

**Páginas responsivas e acessíveis**

**Melhorias de performance (minificação, bundlers iniciais)**

**Adoção de APIs REST para consumir dados**

Começa a separação clara entre frontend e backend: o navegador consome APIs e renderiza views, enquanto o servidor expõe dados.

# HOJE

SPAs, PWAs, frameworks

**SPA: navegação sem recarregar a página (roteamento no cliente)**

SPAs entregam experiência de app.

**Frameworks: React, Vue, Angular (componentes, estado, ecossistema)**

Frameworks trazem componentes reutilizáveis e gestão de estado.

**PWAs: offline, push, instalação via Service Worker**

PWAs aproximam o web app de nativo, com offline e ícone no dispositivo.

**Build moderno: bundlers, transpilers, linters, testes**

# HOJE

Boas práticas e trade-offs

**SSR/SSG e Hydration (Next.js, Nuxt) para SEO e performance**

Para equilibrar SEO/performance com experiência SPA, usamos SSR/SSG.

**Design systems e acessibilidade (ARIA)**

Focar em acessibilidade desde o início.

**Observabilidade: métricas (Core Web Vitals)**

Monitorar LCP/CLS/INP é essencial.

**Trade-offs: complexidade do build, bundle size, segurança**

Atentar a segurança (XSS, CSRF) e ao tamanho do bundle.

SPA pura → ótima UX, mas pode ter bundle grande e SEO pior.

SSR/SSG → melhor SEO/performance inicial, mas aumenta complexidade (servidor/build).

Sempre vigiar segurança (XSS/CSRF) e acessibilidade (ARIA, navegação por teclado).



# O QUE É A WEB?

A **Web** é um sistema para publicar e acessar **recursos** (HTML, CSS, imagens, vídeos, JSON de APIs, etc.) identificados por **URLs** e transferidos via **HTTP** (ou HTTPS, que é HTTP com criptografia TLS).

- **Recurso**

Qualquer coisa endereçável por URL: <https://exemplo.com/artigo>, <https://cdn.com/logo.png>, <https://api.com/users/42>.

- **Como funciona o acesso**

O navegador faz uma requisição HTTP para a URL → recebe resposta com status, headers e corpo (o conteúdo).

- **Hipertexto**

A ideia central é a navegação por links. Cada link aponta para outra URL, formando uma teia (web) de documentos conectados.

- **HTTPS**

Protege confidencialidade e integridade dos dados (cadeadinho na barra do navegador). É o padrão atual.

# O QUE É A WEB?

- **Navegador como cliente universal**

Chrome, Firefox, Safari, Edge entendem os mesmos **padrões** para **renderizar** HTML/CSS, **executar** JavaScript e **falar HTTP/HTTPS**. Isso permite que um site rode em diferentes sistemas e dispositivos sem instalação.

- **Padrões (W3C/WHATWG)**

- **WHATWG** mantém o **HTML Living Standard** (evolução contínua do HTML).
- **W3C** padroniza tecnologias e boas práticas (CSS, acessibilidade/ARIA, etc.).
- O objetivo é **interoperabilidade**: o mesmo código funciona da mesma forma em navegadores diferentes.

- **Web x Internet (diferença-chave)**

- Internet = infraestrutura de rede global (“rede de redes”) e protocolos de transporte/endereçamento (IP, TCP/UDP). Também suporta outros serviços além da Web (ex.: e-mail/SMTP, DNS, FTP, VoIP).
- Web = um serviço que roda em cima da Internet, usando HTTP/HTTPS para transferir recursos identificados por URLs.
- Regra de ouro para a aula: toda Web usa a Internet, mas nem toda Internet é Web (você pode usar a Internet para e-mail, streaming P2P, etc., sem passar por páginas Web).

# O QUE É UMA APLICAÇÃO WEB?

***Aplicação Web** é um software acessado pelo **navegador** (desktop/mobile) através de uma **URL**, sem instalar app nativo. O navegador entende **padrões** (HTML/CSS/JS) e fala com servidores via **HTTP/HTTPS**.*

- **Experiência do usuário (UI/UX)**

Páginas e componentes renderizados no navegador, com responsividade (funciona em telas pequenas e grandes) e acessibilidade (teclado, leitor de tela, contraste).

- **Estado do usuário**

Login/autenticação, preferências, carrinho (em e-commerce), filtros ativos (em dashboards). Parte desse estado pode ficar no cliente (memória/Storage) e no servidor (sessão/DB).

- **Integração com APIs**

A aplicação busca e envia dados por APIs REST (JSON). O frontend orquestra fluxos (listar, criar, editar, apagar), enquanto o backend valida regras, persiste em bancos e garante segurança.

- **Exemplos**

- E-commerce: vitrine → detalhes → carrinho → checkout; integra com APIs de catálogo, estoque, frete e pagamento.
- Dashboard: gráficos e tabelas com filtros, atualizações periódicas; integra com APIs de métricas/relatórios e, às vezes, WebSocket para dados em tempo real.



# RENDERIZAÇÃO & PADRÕES DE INTEGRAÇÃO

- **SSR (Server-Side Rendering)**

O servidor monta o HTML pronto para o primeiro acesso; a página chega rápida e indexável (bom para SEO, landing, catálogo). Depois, o JS “hidrata” e ativa a interatividade. Ex.: vitrine de e-commerce carregando muito rápido com Next.js.

- **CSR (Client-Side Rendering / SPA)**

O servidor envia um shell de HTML + JS; o navegador baixa o bundle e renderiza a UI no cliente, trocando telas sem recarregar a página. Ótimo para apps ricas (dashboards, ferramentas internas). Atenção a bundle size e loading states.

- **Híbrida (SSR + SSG + CSR)**

Páginas estáticas geradas em build (SSG) para conteúdo pouco mutável (ex.: páginas de produto) + SSR para páginas dinâmicas (ex.: busca) + CSR para interações ricas. É o padrão moderno para equilibrar SEO, performance percebida e complexidade.

- **Integração com REST**

- Contrato de dados em JSON (documentado em OpenAPI/Swagger).
- Métodos mapeiam intenções (GET lista/detalhe; POST cria; PUT/PATCH atualiza; DELETE remove).
- Status codes guiam a UI: 200/201/204 (sucesso), 400/401/403/404/409/422 (erros do cliente/validação), 5xx (erros do servidor).
- Boas práticas no frontend: loading/skeleton, cache de GET (SWR/React Query), tratamento de erros com mensagens claras, idempotência para retries de escrita quando suportado, atenção a CORS.

- **Exemplos**

- E-commerce híbrido: SSR/SSG para vitrines e SEO; CSR para carrinho e checkout com validação inline; integra com APIs de preço/estoque/pagamento.
- Dashboard SPA: CSR para filtros instantâneos e gráficos; cache e paginação em GET; possivelmente WebSocket para métricas ao vivo.

# WEB × DESKTOP × MOBILE

| Plataforma              | Instalação                                                              | Portabilidade                                                                          | Acesso a hardware                                                                                                     | Atualização                                                                  | Experiência                                                                                                   |
|-------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Web (navegador)</b>  | Nenhuma (via <b>URL</b> ).<br>(Opcional: “instalar” como <b>PWA</b> )   | <b>Alta</b> (roda em qualquer SO com navegador moderno)                                | <b>Limitado</b> via APIs Web (câmera, microfone, geolocalização, notificações, FS sandbox). Sem acesso profundo ao SO | <b>Imediata</b> (deploy no servidor/CDN; usuários recebem na próxima visita) | Boa se bem otimizada ( <b>SSR/CSR</b> , cache). Sem loja; funciona em múltiplas telas; <b>offline</b> com PWA |
| <b>Desktop (nativo)</b> | Instalador (.msi/.exe, .dmg, .deb/.AppImage); às vezes exige permissões | <b>Média/baixa</b> (build específico por OS; diferenças de distribuição)               | <b>Amplo</b> (FS completo, GPU, USB, drivers; integrações de SO)                                                      | <b>Auto-updater</b> ou manual; dependente do usuário/organização             | <b>Alto desempenho</b> , janelas nativas, atalhos, integração profunda ao SO                                  |
| <b>Mobile (nativo)</b>  | <b>Loja</b> (App Store / Google Play), onboarding e permissões          | <b>Baixa</b> entre plataformas (iOS/Android); <i>cross-platform</i> ajuda (RN/Flutter) | <b>Amplo</b> (GPS, câmera, sensores, push, background tasks, biometria)                                               | Via loja (review/aprovação), <i>rollout</i> gradual                          | UX nativa (gestos, padrões de plataforma), <b>offline</b> sólido, <b>push</b> confiável                       |

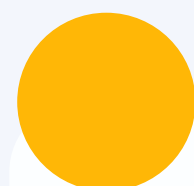
# POR QUE FRONTEND IMPORTA?

Primeira impressão e usabilidade



## Primeira impressão em 50-500 ms

A primeira impressão define se o usuário continua ou abandona. Layout limpo + texto claro reduzem carga cognitiva.



## Clareza visual

Usabilidade = fricção baixa: formulários curtos, feedback imediato, erros explicados. (hierarquia, contraste, espaço em branco)



## Fluxos simples

Impacto de negócio: reduz bounce rate, aumenta engajamento no 1º contato, melhora NPS. (menos cliques, menos campos)



## Padrões consistentes

Métricas: Bounce %, tempo na página, taxa de conclusão de fluxo (ex.: cadastro), CSAT/NPS.

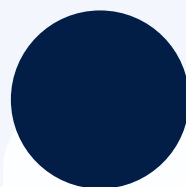
# POR QUE FRONTEND IMPORTA?

Performance percebida



## Velocidade percebida > só tempo real

Mesmo com backends rápidos, o usuário sente lentidão se a tela não dá sinais de vida.



## Carga progressiva

Skeletons, pré-busca de dados e lazy loading fazem o app “parecer” rápido.



## Otimização de bundle

Reduzir bundle (code splitting, remoção de dependências pesadas) e uso de cache melhoram a resposta.

Impacto de negócio: cada segundo a menos tende a elevar conversão e retenção; menos suporte por “site travado”.

Métricas: tempo até conteúdo visível, tempo até interatividade, tamanho do bundle, taxa de erro de UI.

# POR QUE FRONTEND IMPORTA?

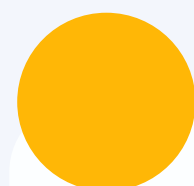
Acessibilidade e alcance

Requisição → Resposta (URL, método, cabeçalhos, corpo)



## Semântica (tags corretas) e teclado

Acessibilidade amplia o público (pessoas com deficiências, conexões ruins, dispositivos diversos).



## Contraste e tamanho de fonte

Beneficia todos: melhor navegação por teclado, leitura em telas pequenas, leitores de tela.



## Textos alternativos (imagens), rótulos de campos

Impacto de negócio: mais alcance e mercados, menor risco legal, melhor reputação de marca.



## Internacionalização e preferências do usuário

Métricas: taxa de uso via teclado, verificações automáticas (linters/auditorias), % de páginas com contraste adequado, feedbacks de suporte.



# POR QUE FRONTEND IMPORTA?

Conversão e retenção



**Foco no objetivo (CTA claro e visível)**

Converter = guiar o usuário: CTA destacado, etapa a etapa sem surpresas.



**Redução de fricção**

Validação inline evita refazer formulários; microcópias reduzem ansiedade (“Dados protegidos”, “Você pode mudar depois”).



**Confiança**

Retenção cresce com experiências estáveis, rápidas e com reengajamento (push de PWA, salvar no home).



**Loop de retorno**

Impacto de negócio: ↑ taxa de conversão (cadastro/compra), ↓ churn, ↑ LTV.  
Métricas: conversão por etapa do funil, abandono de carrinho/formulário, retorno 7/30 dias, MAU/DAU

CTA ou Chamada para Ação, refere-se a um elemento de design em um site ou aplicativo que induz o usuário a realizar uma ação específica, como clicar em um botão para comprar, preencher um formulário, assistir a um vídeo ou se inscrever em uma newsletter.

# POR QUE FRONTEND IMPORTA?

Amarrando aos objetivos de negócio



**O frontend traduz estratégia em experiência**

Cada decisão de UI deve ter hipótese de impacto (ex.: “CTA mais visível aumenta cadastro em 10%”).



**Hipóteses → Experimentos → Métricas**

Rodar A/B tests e acompanhar métricas fecha o ciclo de melhoria contínua.



**Alinhamento com produto, marketing e engenharia**

Trabalhar com marketing (mensagem/SEO), produto (prioridades do funil) e engenharia (viabilidade, performance) garante execução.

Quadro de métricas sugerido:

- Aquisição: CTR de landing, bounce;
- Ativação: conversão no primeiro fluxo;
- Retenção: retorno 7/30 dias, churn;
- Receita: taxa de checkout, ticket médio;
- Qualidade: erros de UI, tempo de resposta percebido.



# O QUE É UM SISTEMA DISTRIBUÍDO?

“Distribuído” = partes do sistema executam em lugares distintos e cooperam pela rede. Diferente de um app monolítico em uma única máquina.

**Múltiplos componentes rodando em máquinas diferentes**

**Escalabilidade: vertical vs. horizontal (load balancer)**

Vertical = máquina mais potente; Horizontal = mais réplicas atrás de um balanceador. Horizontal é a base da nuvem moderna.

Rede é não confiável por natureza: há latência (demora), perdas e partições (quando partes não se veem). Por isso, todo chamado remoto precisa de timeout e repetição com backoff (ex.: exponencial).

**Comunicação via rede (latência, perda, partição)**

**Tolerância a falhas: redundância, retry + timeout**

Assumir que falhas acontecem (máquina cai, rede oscila). Técnicas: replicação, circuit breaker, health checks, auto-restart.

Concorrência: várias requisições simultâneas requerem controle de acesso a dados e evitar condições de corrida; filas e locks podem ser necessários.

**Concorrência: muitos usuários/processos ao mesmo tempo**

**Consistência: forte vs. eventual (trade-offs de CAP)**

Em sistemas distribuídos, às vezes escolhemos eventual consistency (propaga em segundos) para ganhar disponibilidade e tolerância a partição (intuição do CAP). Guarde o termo; volta no cliente-servidor.



# COMUNICAÇÃO E DESENHO (CLIENTE ↔ API ↔ SERVIÇOS/BANCOS)

HTTP é o alicerce da Web: simples, interoperável. Em APIs REST, cada URL representa um recurso (“/users/42”)

**Protocolos: HTTP (REST), gRPC, filas (mensageria)**

**Contrato: JSON + documentação (OpenAPI/Swagger)**

Formato de dados: geralmente JSON; o “contrato” da API (campos obrigatórios, tipos) é descrito com OpenAPI/Swagger, facilitando testes e clients automáticos.

Usamos verbos para intenção (GET lê, POST cria, PUT/PATCH altera, DELETE remove). Status codes comunicam resultado (200/201 sucesso, 400 erro do cliente, 500 erro do servidor).

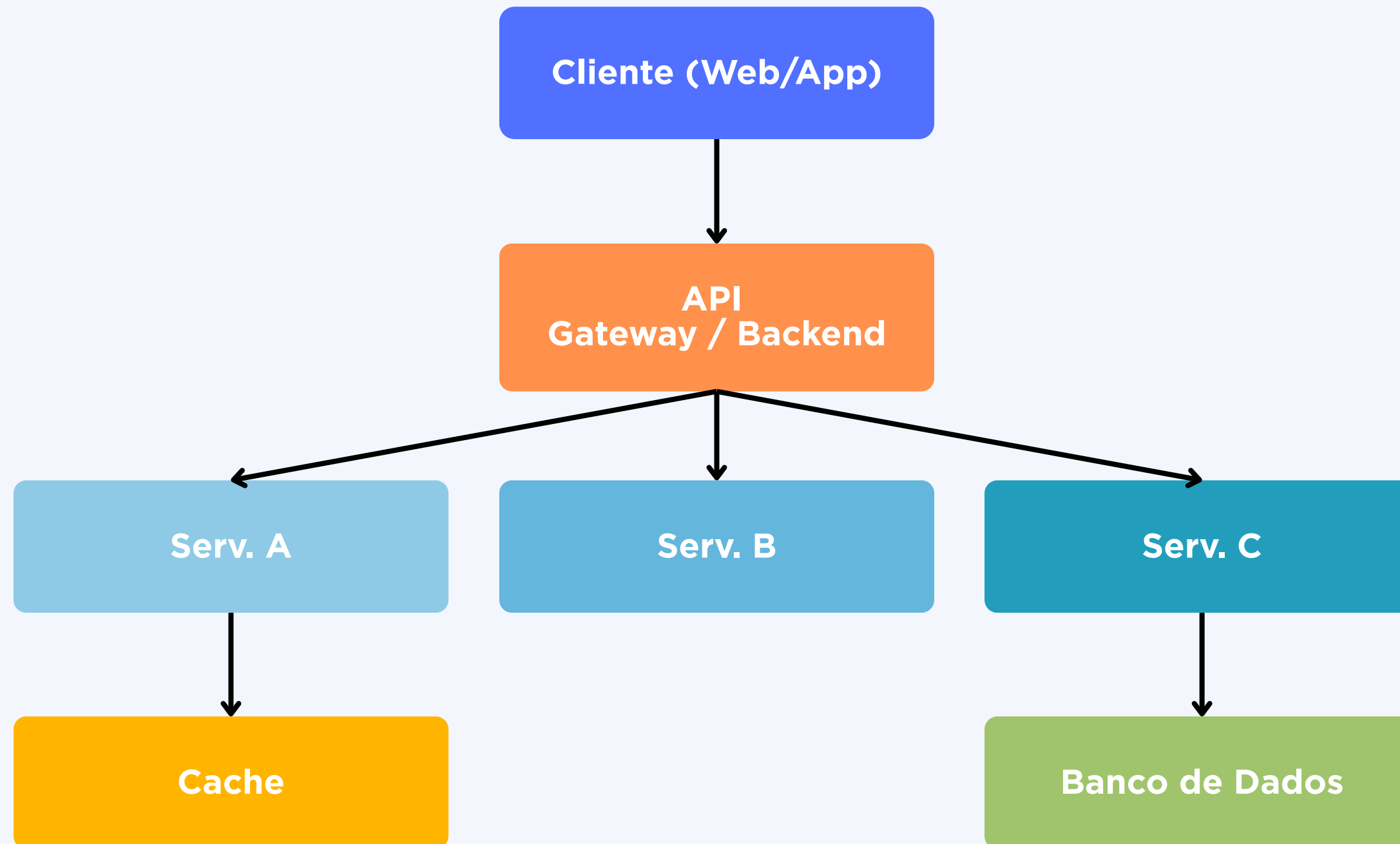
**HTTP/REST: verbos (GET/POST/PUT/DELETE), códigos (2xx/4xx/5xx)**

**Padrões: idempotência, retry, circuit breaker**

Boas práticas de resiliência:

- Idempotência em operações de escrita (ex.: idempotency key), para que retries não dupliquem efeitos.
- Timeout + Retry com backoff e circuit breaker para evitar cascata de falhas.
- Cache (ex.: Redis/CDN) para aliviar cargas e reduzir latência percebida.

# COMUNICAÇÃO E DESENHO (CLIENTE ↔ API ↔ SERVIÇOS/BANCOS)



# REQUISIÇÃO/RESPOSTA + MÉTODOS

- **Requisição → Resposta (URL, método, cabeçalhos, corpo)**

O navegador (cliente) envia uma requisição HTTP para uma URL com um método (verbo), cabeçalhos (metadados) e, às vezes, um corpo (dados). O servidor responde com status, headers e opcionalmente corpo.

- **Métodos**

- GET: ler dados (sem corpo ou com corpo ignorado).
- POST: criar/acionar; corpo geralmente JSON.
- PUT: substituir um recurso inteiro.
- PATCH: alterar parcialmente um recurso.
- DELETE: remover recurso.

- **Segurança/semântica**

- **Safe** = não altera estado (**GET**).
- **Idempotente** = repetir gera o mesmo efeito (**GET, PUT, DELETE**; **POST** não é). Isso importa para **retries** e **cache**.

- **Headers-chave**

- Content-Type: application/json (tipo do corpo que você envia).
- Accept: application/json (tipo que você quer receber).
- Authorization: Bearer <token> (autenticação).

- **Params**

- Path params: fazem parte da rota (/users/42).
- Query params: filtros/paginação (?page=2&limit=20).

```
await fetch('/api/users/42', {
  method: 'PATCH',
  headers: {'Content-Type': 'application/json', 'Accept': 'application/json'},
  body: JSON.stringify({ name: 'Ana' })
})
```

# STATUS

- **Status**

- **200** OK (resposta com corpo)
- **201** Created (recurso criado; às vezes com Location)
- **204** No Content (sucesso sem corpo)
- **400** Bad Request (payload inválido)
- **401** Unauthorized (falta/expirou token)
- **403** Forbidden (token não permite)
- **404** Not Found (rota/ID não existe)
- **409** Conflict (conflito de versão/estado)
- **422** Unprocessable Entity (erros de validação campo a campo)
- **500** (erro no servidor).

No frontend, mapeie cada um para mensagens claras e estados de UI (ex.: inline errors).

# ESSENCIAIS DE FRONTEND (CORS, CACHE, COOKIES)

- **CORS (Cross-Origin Resource Sharing)**

Navegador só permite chamar domínios diferentes se o servidor responder com headers CORS (Access-Control-Allow-Origin, etc.). Se der erro de CORS no console, é configuração do servidor, não do fetch.

- **Cache (percepção de velocidade)**

- Cache-Control: max-age/s-maxage controlam quanto tempo respostas podem ser reutilizadas.
- Validadores: ETag + If-None-Match → o servidor responde 304 Not Modified e o browser usa o cache.
- Em SPAs, combine cache de GET com revalidação (ex.: SWR/React Query).

- **Autenticação**

- Cookies (podem ser HttpOnly, SameSite, Secure) — úteis para SSR, mas atenção a CORS e CSRF.
- Bearer tokens (header Authorization) — simples no frontend;

- **Content negotiation**

- Sempre peça/retorne JSON (Accept, Content-Type).



# ARQUITETURA CLIENTE-SERVIDOR



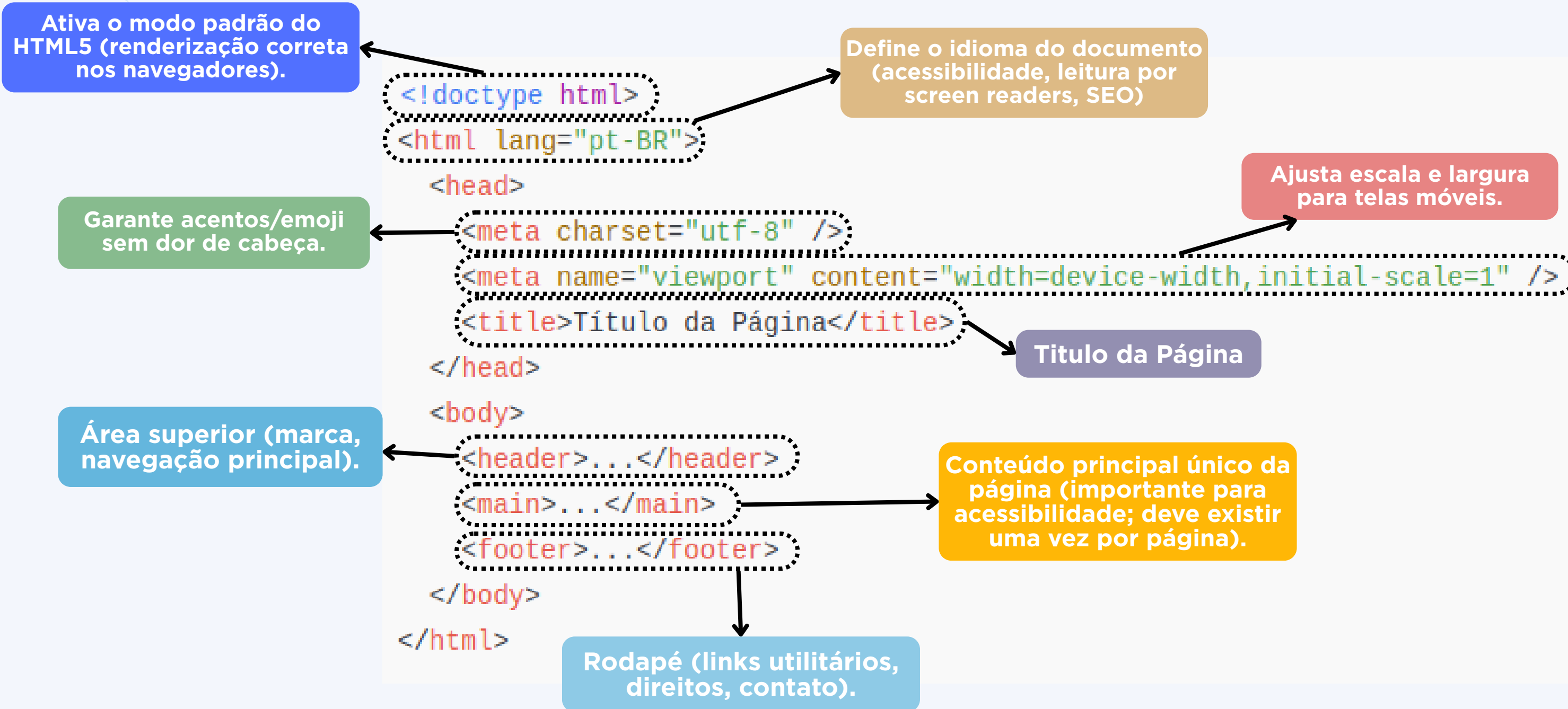
- **Frontend (Browser)**

- Renderiza UI (HTML/CSS/JS), navegação, estado de tela
- Chama APIs (fetch/Axios), trata status e erros
- Validação de UX (campos, máscaras), acessibilidade, cache do navegador

- **Backend (API REST)**

- Regra de negócio, persistência (DB), segurança (authZ/authN)
- Contrato de dados (JSON, OpenAPI), versionamento
- Observabilidade (logs, métricas, tracing), rate limit e políticas

# HTML5: ESTRUTURA MÍNIMA



# BOAS PRÁTICAS HTML

- **Semântica = significado**

- Use header, nav, main, section, article, aside, footer, figure/figcaption, time, etc. Isso melhora acessibilidade (leitores de tela entendem a estrutura), SEO (crawlers entendem relevância) e manutenção (código legível).
- Evite abusar de <div>/<span> quando houver tag semântica apropriada.

- **Títulos hierárquicos**

- Um h1 por página, seguido de h2 para seções, h3 para subseções (sem “pular” níveis).
- Bom: h1 “Blog” → h2 “Artigos recentes” → h3 “Artigo X”.
- Evite usar h\* só para “ficar grande”; estilo é no CSS.

- ***alt* em imagens**

- Descreva a função da imagem.
- Conteúdo informativo: alt="Mapa das lojas no Nordeste"
- Decorativa: alt="" (vazio) para o leitor de tela ignorar.
- Não repita “imagem de...”, e não coloque texto gigante no alt.

- ***Forms com label + name***

- label associado (via for="id" ou envolvendo o input) melhora foco/teclado/voz.
- name é obrigatório para o backend receber os dados e para autofill do navegador.

```
<label for="email">E-mail</label>  
<input id="email" name="email" type="email" autocomplete="email" required>
```



# BOAS PRÁTICAS HTML

- **Atributos base:**

- lang="pt-BR" no <html> ajuda pronúncia e segmentação de idioma (A11y + SEO).
- dir="rtl" quando idioma for direita→esquerda (árabe/hebraico).

- **Meta essenciais**

- <meta charset="utf-8"> para acentuação/emoji sem problemas.
- <meta name="viewport" content="width=device-width, initial-scale=1"> para mobile-first e layout responsivo.
- <meta name="description" content="..."> gera snippet nos buscadores (até ~160 caracteres, ação/benefício claros).
- Extras úteis: color-scheme="light dark", theme-color, ícones (<link rel="icon">).

- ***Por que isso impacta SEO/A11y***

- SEO: semântica correta + títulos hierárquicos + description → indexação melhor e CTR maior.
- Acessibilidade: semântica + alt + label/name + idioma correto → leitura assistiva fluida, navegação por teclado, menor taxa de abandono por frustração.

- ***Checklist rápido***

- Tem lang e charset?
- Tem viewport para mobile?
- Existe um h1 e a hierarquia está correta?
- Imagens informativas têm alt adequado (decorativas alt="")?
- Inputs têm label vinculado e name?
- Há meta description significativa?

# TAGS SEMÂNTICAS PRINCIPAIS

| Tag       | Para que serve                                                 | Exemplo                                                          |
|-----------|----------------------------------------------------------------|------------------------------------------------------------------|
| <header>  | Cabeçalho da página ou de uma seção (logo, título, nav local)  | html<br><header><h1>Blog</h1></header>                           |
| <nav>     | Conjunto de links de navegação (principal, secundária)         | html<br><nav aria-label="Principal"><a href="/">Início</a></nav> |
| <main>    | <b>Conteúdo principal único</b> da página (1× por página)      | html<br><main><h1>Artigos</h1>...</main>                         |
| <section> | Agrupa conteúdo <b>temático</b> com título                     | html<br><section><h2>Tutoriais</h2>...</section>                 |
| <article> | Conteúdo <b>independente/reutilizável</b> (post, card, review) | html<br><article><h2>Post X</h2>...</article>                    |
| <aside>   | Conteúdo complementar (lateral, notas, promo)                  | html<br><aside>Assine a newsletter</aside>                       |
| <footer>  | Rodapé da página ou de uma seção (créditos, contatos)          | html<br><footer>© 2025</footer>                                  |

# TAGS SEMÂNTICAS PRINCIPAIS

## Section vs Article — quando usar

- article: faz sentido sozinho (pode ser syndicado/embutido noutro lugar). Exemplos: post de blog, card de produto, review, issue.
- section: bloco temático dentro da página que depende do contexto; normalmente tem título próprio (h2/h3) e agrupa subtópicos.
- Regra prática: “Se eu recortar e colar em outro site, ainda faz sentido?” → Sim = article, Não = section.

## Landmarks e acessibilidade

- Um main por página. header/footer podem existir no documento e dentro de article/section (cabeçalhos/rodapés locais).
- Use nav[aria-label=”...”] quando houver várias navegações (ex.: “Principal”, “Rodapé”).
- Cada section deve ter um título (geralmente h2). Evite section sem heading — use <div> se não houver um tópico claro.

## Hierarquia de títulos

- 1× h1 para o título da página. Depois h2 para seções, h3 para subseções, sem “pular” níveis apenas por estilo (estilo é CSS).
- Dentro de um article, o título principal desse artigo pode ser h2 (ou o próximo nível adequado).

# TAGS SEMÂNTICAS PRINCIPAIS

## SEO & UX

- Semântica correta ajuda motores de busca a entender o que é o quê (navegação, conteúdo principal, artigos).
- Melhora a navegação por teclado/leitores de tela e a descoberta (ex.: rich snippets quando combinada com dados estruturados, se aplicável).

## Erros comuns (evitar)

- Usar section só para “dar margem” — se não tiver título/tema, use <div>.
- Colocar mais de um main.
- nav com qualquer grupo de links (ex.: links dentro de um artigo) — nav é para navegação da página/área.
- Títulos fora de ordem (pular de h1 para h4).

# TAGS COMUNS E ATRIBUTOS

- **p vs span:**

- p é bloco (parágrafo, separa ideias);
- span é inline para trechos dentro do parágrafo.

```
<p>Texto de parágrafo.</p>  
<span class="destaque">inline</span>
```

- **strong/em:**

- significado semântico (importância/ênfase) — melhor para A11y/SEO que só “deixar em negrito/itálico” via CSS.

```
<strong>importante</strong> <em>ênfase</em>
```

- **Link (mesma aba)**

```
<a href="/contato">Fale conosco</a>
```

- **Link (nova aba, seguro)**

```
<a href="https://exemplo.com"  
  target="_blank" rel="noopener noreferrer">Site</a>
```



# MÍDIA & LISTAS

- Imagem acessível

```

```

- Imagem decorativa

```

```

- Vídeo com controles/legenda

```
<video controls width="640" preload="metadata">  
  <source src="/demo.mp4" type="video/mp4">  
  <track kind="captions" src="/legendas.vtt" srclang="pt-BR" label="PT-BR">  
</video>
```

- Listas (ordenada e não-ordenada)

```
<ul><li>Item A</li><li>Item B</li></ul>  
<ol><li>Passo 1</li><li>Passo 2</li></ol>
```

# FORMULÁRIOS ESSENCIAIS

- Tipos de input, required, placeholder

```
<form>
  <label for="email">E-mail</label>
  <input id="email" name="email" type="email" required />
  <button type="submit">Enviar</button>
</form>
```

# EVENTOS (VISÃO HTML/DOM)

- Atributo HTML (onclick) × addEventListener (melhor prática)

```
<button id="ok">OK</button>
<script>
  document.getElementById('ok').addEventListener('click', () => alert('OK'));
</script>
```



# REQUISITOS DO PROJETO (ENTREGA EM GRUPO)

## **Stack obrigatória:**

- Backend (Node.js + MongoDB)
- Frontend (React)

## **Formação de equipe:**

- 4-5 pessoas por grupo

## **Funcionalidades mínimas**

- CRUD completo de pelo menos 2 entidades relacionadas (ex.: Usuário ↔ Itens)
- Autenticação (login/logout) e autorização (acesso por perfil)
- Validações no backend (campos obrigatórios, formatos)
- Listagem com filtro/paginação no frontend

## **API REST**

- Endpoints padronizados (/api/v1/...) com status codes corretos
- Documentação (OpenAPI/Swagger) acessível na rota /docs

## **Frontend (React)**

- UI responsiva, rotas (React Router), estados de carregamento/erro
- Formulários com validação e mensagens de erro claras

## **Qualidade**

- README completo (setup, scripts, decisões técnicas, endpoints)
- Commits frequentes e mensagens claras (todos os membros)

## **Entrega & Apresentação**

- Repositório Git (monorepo ou dois repos) + deploy (ou vídeo demo)
- Apresentar no dia: live demo (~8-10 min) + arquitetura + lições aprendidas